

PROJECT 1 : SOCKET PROGRAMMING

MOEP

Mini Order Exchange Protocol

การออกแบบโปรแกรมและโปรโตคอล

TCP สำหรับคำสั่งซื้อขาย + **UDP** สำหรับข้อมูลตลาด

ปิยากร ผลพานิช | 6710451054 | 01418351

1. MOEP คืออะไร
2. คุณลักษณะของ Application
3. เลือก Transport Layer แบบใด
4. ออกแบบ Application-Layer Protocol
5. Matching Engine
6. สถาปัตยกรรมและ Demo
7. สรุป

ประเด็นหลักของงานนี้: เลือก transport ตาม *ประเภทของ traffic* ไม่ใช่ตาม *ตัวแอป*

- ตลาดหลักทรัพย์จำลองแบบ **client-server**: exchange server 1 ตัว + trader client หลายตัว
ซื้อขายสัญลักษณ์เดียว (AAPL)
- Flow สั้น ๆ
 - Trader ต่อ **TCP** → ส่ง REGISTER ครั้งเดียว
 - ส่ง ORDER_NEW ได้ไม่จำกัด โดยไม่ต้อง register ใหม่
 - Server จับคู่ตาม price-time priority
 - กระจายผลกลับให้ทุกคนทาง **UDP**

เลือก transport ต่อ “ประเภทของ traffic” ไม่ใช่ต่อ “ตัวแอป”

- MOEP มี **role เดียว: Trader** ซึ่งทำ 2 หน้าที่พร้อมกันในคนเดียว คือ ส่งคำสั่ง (TCP) และ รับ market data (UDP)
- ต่างจากงานที่มี Host/Voter/Viewer แยกกัน ตารางข้อความของ MOEP จึงต้อง แยกตาม **ทิศทางการสื่อสาร** ไม่ใช่แยกตาม role:
 1. Client → Server (TCP)
 2. Server → Client (TCP เฉพาะ connection นั้น)
 3. Server → Clients ทั้งหมด (UDP fan-out)
- Server เจ้าเดียวถือ order book กลาง เป็น **ความจริงหนึ่งเดียว**

- **Many-to-one-to-many**

trader หลายคนส่ง order เข้า server ตัวเดียว แล้ว server กระจายสถานะตลาด กลับไปให้ทุกคนพร้อมกัน

- **Central authoritative state**

order book กลางเป็นความจริงหนึ่งเดียว client ไม่เก็บ state เอง แค่สะท้อนสิ่งที่ server บอก

- **Continuous, session-persistent**

ไม่มีรอบ, connection ค้างไว้ตลอด session, REGISTER ครั้งเดียวใช้ได้ตลอด

- **Dual-channel ต่อ client**

trader แต่ละคนมี 1 TCP connection + 1 UDP socket ของตัวเอง (UDP port เป็น ephemeral ที่ OS สุ่มให้)

- **Self-healing market data feed**

ตรวจจับ packet หายจาก sequence number แล้วกู้คืนเองผ่าน TCP โดยผู้ใช้ไม่ต้องทำอะไร

- **หนึ่ง client library รองรับ 2 หน้า UI**

terminal CLI (`client.py`) และ web dashboard (`web_app.py`) ใช้ protocol เดียวกันทุกตัวอักษร ไม่มีการแก้ wire format เพื่อรองรับเว็บ

โปรแกรมนี้เลือกใช้ ทั้ง TCP และ UDP — แยกตามประเภทของ traffic ไม่ใช่ตามตัวแอป

TCP

- connection-oriented
- reliable + in-order
- มี flow / congestion control
- มี handshake overhead
- head-of-line blocking

UDP

- connectionless
- best-effort (ไม่การันตี)
- low overhead
- sendto() ไม่ block
- เหมาะกับ fan-out ความถี่สูง

	Order Entry	Market Data
ทิศทาง	1 client → server	server → N clients
ความถี่	ต่ำ (ตามที่คนกด)	สูง (ทุกครั้งที่ book ขยับ)
ข้อมูลหายได้ไหม	ยอมไม่ได้ — เป็นเงินจริง	ยอมได้ชั่วคราว ขอแค่กู้คืนได้
ลำดับสำคัญไหม	สำคัญมาก (price-time priority)	สำคัญน้อยกว่า เอาค่าล่าสุด

traffic คนละแบบ มีข้อกำหนดคนละอย่าง จึงไม่ควรบังคับให้ใช้ transport เดียวกัน

- **Correctness = เงิน**

order หายหรือซ้ำคือ bug ทางการเงิน ไม่ใช่แค่ UX bug

- **In-order delivery**

ลำดับที่ order มาถึงคือสิ่งที่ตัดสิน price-time priority ถ้าลำดับสลับ ผลการจับคู่ผิดทันที

- **Connection state ใช้ประโยชน์ได้**

connection ที่ค้างไว้ = ตัวตนของ trader คนนั้น ไม่ต้องส่ง token ทุกครั้ง และรู้ทันทีเมื่อ client หลุด

ถ้าใช้ UDP กับ order จะต้องเขียน ACK, retransmit, dedup, sequencing เองทั้งหมด = สร้าง TCP ขึ้นมาใหม่โดยไม่จำเป็น

- หลีกเลียง **head-of-line blocking**

ถ้าใช้ TCP แล้ว update #41 หาย update #42, #43 ที่มาทีหลังจะค้างรอด้วย ทั้งที่ไม่จำเป็น

- **sendto()** ไม่ block

server ยิง market data ให้ N client โดยไม่ต้องรอ ACK จากใครเลย ไม่ทำให้ thread ที่กำลังจับคู่ order ค้าง

- ต่อยอดเป็น **multicast** ได้ในอนาคต (TCP ทำไม่ได้) — แต่ปัจจุบัน ยังเป็นการวน sendto() ที่ละ client คือ unicast fan-out ไม่ใช่ IP broadcast จริง

ไม่ได้เลือกระหว่าง “เร็ว” กับ “แม่นยำ” — แต่เอาความแม่นยำไปวางตรงที่มันจำเป็น และเอาความเร็วไปวางตรงที่มันได้ผล

- ปัญหา: UDP ทำ packet หายได้จริง client จะเห็น seq กระโดด (expected=41, got=43)
- **Sequence numbering**
ทุก UDP message (MARKET_DATA และ TRADE) ใช้ตัวนับ seq ตัวเดียวกัน เรียงต่อเนื่อง client จึงตรวจได้ทันทีว่าขาด
- **Gap fill ผ่าน channel ที่เชื่อถือได้**
client ส่ง GAP_FILL_REQUEST(from_seq, to_seq) ทาง **TCP** → server ค้นจาก history buffer (เก็บ 200 message ล่าสุด) แล้วส่งกลับของเดิมทุกตัว
- ถ้า seq ที่ขอหลุดจาก history ไปแล้ว → ตอบ 410 GONE พร้อม missing_seq

ความน่าเชื่อถือไม่ได้มาจาก transport เดียว — แต่มาจากการออกแบบให้สอง transport ทำงานร่วมกัน

12 Framing และ Port

■ Framing

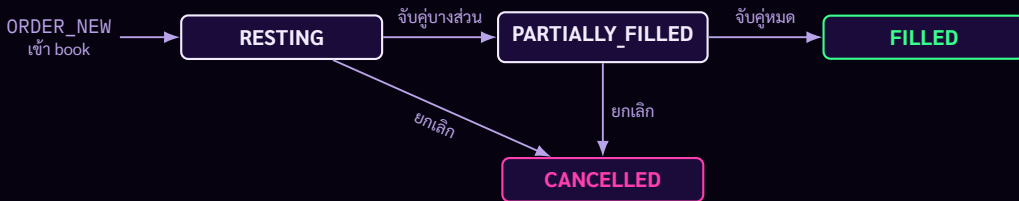
- TCP: 1 message ต่อ 1 บรรทัด JSON คั่นด้วย \n (byte stream ไม่มีขอบเขต message ในตัวเอง)
- UDP: 1 datagram ต่อ 1 JSON message (UDP รักษาขอบเขตให้อยู่แล้ว ไม่ต้องมีตัวคั่น)

■ Port

- TCP fixed 5000 — trader ทุกคนต่อ port เดียวกัน
- UDP ไม่มี port กลาง — client แต่ละคน bind port ของตัวเอง (OS สุ่มให้) แล้วบอก server ผ่าน field udp_port ตอน REGISTER

```
SESSION (client REGISTER ครั้งเดียว)
|-- ORDER_NEW    -> matching -> broadcast UDP
|-- ORDER_NEW    -> matching -> broadcast UDP
|-- ORDER_CANCEL
`-- (UDP feed ทำงานตลอด, มี GAP_FILL เมื่อ seq ขาด)
```

13 State Diagram ของ Order



order ที่ FILLED หรือ CANCELLED แล้ว ยกเลิกซ้ำไม่ได้
server ตอบ **404 NOT_FOUND**

14 รูปแบบข้อความ: Client → Server (TCP)

Register

ครั้งแรกเดียวตอนเข้าตลาด

```
{"type": "REGISTER", "udp_port": <int>}
```

New order

ส่งซ้ำได้ไม่จำกัดโดยไม่ต้อง register ใหม่

```
{"type": "ORDER_NEW", "symbol": "AAPL", "client_order_id": <str>,  
  "side": "BUY" | "SELL", "price": <float>, "qty": <int>}
```

Cancel order

ยกเลิกคำสั่งที่ยังไม่ถูกจับคู่หมด

```
{"type": "ORDER_CANCEL", "client_order_id": <str>}
```

Gap fill

ขอ market data ที่หายไปคืน

```
{"type": "GAP_FILL_REQUEST", "from_seq": <int>, "to_seq": <int>}
```

Register ack

client_id ถาวรใช้ทั้ง session

```
{ "type": "REGISTER_ACK", "status_code": 200, "status_phrase": "OK",  
  "client_id": <str>, "symbol": "AAPL" }
```

Order ack (matched)

จับคู่เต็มจำนวนทันที

```
{ "type": "ORDER_ACK", "status_code": 201, "status_phrase": "MATCHED", "order_id": <int>,  
  "filled_qty": <int>, "remaining_qty": 0, "trade_count": <int> }
```

Order ack (partial)

จับคู่บางส่วน ที่เหลือขึ้น book

```
{ "type": "ORDER_ACK", "status_code": 202, "status_phrase": "PARTIAL_FILL", ... }
```

Cancel ack

ยกเลิกสำเร็จ

```
{ "type": "CANCEL_ACK", "status_code": 204, "status_phrase": "CANCELLED", "order_id": <int> }
```

Gap fill response

ส่งของเดิมที่หายกลับมาทั้งชุด

```
{ "type": "GAP_FILL_RESPONSE", "status_code": 200, "messages": [...] }
```

16 รูปแบบข้อความ: Server → Clients ทั้งหมด (UDP fan-out)

Market data

ส่งทุกครั้งที่ order book เปลี่ยน

```
{"type": "MARKET_DATA", "seq": <int>, "best_bid": <float>, "best_ask": <float>,  
  "bids": [{ "price": ..., "qty": ... }], "asks": [{ "price": ..., "qty": ... }]}
```

เป็น snapshot เต็ม top-5 ไม่ใช่ delta — client ที่เพิ่งเข้ามาหรือเพิ่งพลาด packet เห็นภาพตลาดครบถ้วนที่จาก message เดียว Trade

ส่งเมื่อเกิดการจับคู่สำเร็จ

```
{"type": "TRADE", "seq": <int>, "price": <float>, "qty": <int>, "side": "BUY" | "SELL" }
```

MARKET_DATA และ TRADE ใช้ seq ชุดเดียวกัน เพื่อให้ client ตรวจ gap ได้จาก stream เดียว

Code	Phrase	ความหมาย
200	OK	สำเร็จทั่วไป (register / order วางบน book)
201	MATCHED	จับคู่เต็มจำนวนทันที
202	PARTIAL_FILL	จับคู่บางส่วน ที่เหลือค้าง book
204	CANCELLED	ยกเลิกสำเร็จ
400	BAD_REQUEST	JSON ผิดรูป / side ไม่ถูกต้อง / message type ไม่รู้จัก
403	REJECTED	ราคาหรือจำนวนไม่ถูกต้อง (≤ 0)
404	NOT_FOUND	ยกเลิก order ที่ไม่มีอยู่แล้ว
410	GONE	gap-fill ขอ seq ที่หลุดจาก history buffer ไปแล้ว
500	INTERNAL_ERROR	กำหนดไว้ใน spec แต่ยังไม่มี code path เรียกใช้จริง

```
Client1 -TCP-> {"type":"REGISTER","udp_port":5001}
Server   -TCP-> {"status_code":200,"status_phrase":"OK","client_id":"c_101"}

Client2 -TCP-> {"type":"REGISTER","udp_port":5002}
Server   -TCP-> {"status_code":200,"status_phrase":"OK","client_id":"c_102"}

Client1 -TCP-> {"type":"ORDER_NEW","side":"BUY","price":214.41,"qty":100}
Server   -TCP-> {"status_code":200,"status_phrase":"OK","order_id":1}
Server   -UDP-> MARKET_DATA seq=1 bids=[{"214.41",100}] asks=[]

Client2 -TCP-> {"type":"ORDER_NEW","side":"SELL","price":214.41,"qty":100}
Server   -TCP-> {"status_code":201,"status_phrase":"MATCHED","order_id":2}
Server   -UDP-> TRADE seq=2 price=214.41 qty=100
Server   -UDP-> MARKET_DATA seq=3 bids=[] asks=[]
```

```
Client1 <-- UDP: seq=1, seq=3      (seq=2 หายกลางทาง)
Client1 expected=2, got=3          --> GAP DETECTED

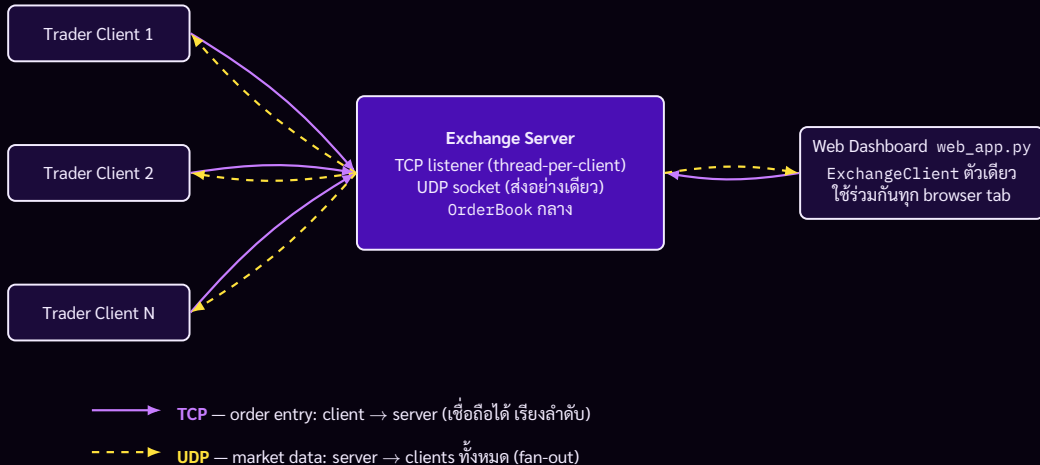
Client1 -TCP-> {"type":"GAP_FILL_REQUEST","from_seq":2,"to_seq":2}
Server  -TCP-> {"type":"GAP_FILL_RESPONSE","status_code":200,
               "messages":[{"type":"TRADE","seq":2, ...}]}
```

```
Client1 [recovered]  stream ต่อเนื่องครบ 1, 2, 3
```

การกู้คืนวิ่งบน **TCP** ทั้งหมด — ฝั่ง UDP ไม่ต้องรู้เรื่องอะไรเลย

- **ราคาดีที่สุ่ดก่อนเสมอ** ราคาเท่ากันตัดสินด้วยลำดับที่มาถึง (time priority)
- **ลำดับเวลาที่ใช้ = ลำดับที่ TCP ส่งมอบให้**
เชื่อมกลับไปหาเหตุผลที่เลือก TCP ในสไลด์ 9: ถ้า transport ส่งมอบไม่เรียงลำดับ price–time priority จะพังทันที
- **โครงสร้างข้อมูล:** 2 heap — buy_heap เก็บราคาติดลบเพื่อให้ กลายเป็น max-heap, sell_heap เป็น min-heap ปกติ → หา best price ได้ที่ $O(\log n)$
- **Lazy deletion สำหรับ cancel:** แค่เปลี่ยน status เป็น CANCELLED ไม่ลบออกจาก heap จริง แล้วข้ามทิ้งตอน pop เจอ

21 Architecture



Web dashboard เป็นเพียงชั้น UI บน client library เดิม — ไม่แก้ wire protocol แม้แต่บรรทัดเดียว

- **ยิง seed order 10–15 ตัวก่อนอัปเดตวิดีโอ**

อย่าปล่อย order book วางตอนเริ่ม demo เพราะจะดูเหมือนโปรแกรมยังไม่ทำงาน

- **ต้องเห็น status code + status phrase ทั้งสองฝั่ง**

terminal ของ server.py / client.py พิมพ์อยู่แล้วผ่าน log() เช่น

```
[SERVER] SEND ORDER_ACK status=201 MATCHED
```

(payload จริงเป็น JSON ไม่ใช่ pipe-delimited แต่ log บรรทัดนี้อ่านง่ายพอสำหรับวิดีโอ)

- **โชว์ gap recovery สด ๆ**

Dashboard มี Protocol Message Log + gap-detected banner อยู่แล้ว ตั้งใจ trigger gap จริง

ถ้าทำได้ (ดีเลย์ / drop UDP ชั่วคราว) จะได้เห็น GAP DETECTED → GAP_FILL_REQUEST

- แบ่ง transport **ตามประเภทของ traffic** ไม่ใช่ตามตัวแอป: **TCP** สำหรับ order, **UDP** สำหรับ market data
- **TCP** เพราะ order ต้องถูกต้องและเรียงลำดับ — ความถูกต้องคือเงิน
- **UDP** เพราะ price feed ต้องสด และไม่ยอมให้ packet เก่าบล็อก packet ใหม่
- จุดที่ UDP หายจริง ก็คืนด้วย **sequence number + GAP_FILL** ผ่าน **TCP** — ได้ best of both worlds ไม่ใช่เลือกข้างใดข้างหนึ่งแล้วรับผลเสียทั้งหมด
- ยืนยันด้วย **automated test 5 scenario + live dashboard**

Thank You for Listening